



UNIVERSITÀ
DI PISA

Artificial Intelligence and Data Engineering

Distributed Systems And Middleware Technologies

Project Report

Title

Entertainment Ticket Reservation System

Group Members

Van Khai Do

Mohammed Ragab Elsinousi Abdalhady

2025/2026

Abstract.....	1
1. Introduction.....	2
1.1. Project Compliance with Course Requirements.....	2
2. Objectives.....	2
3. Problem Statement.....	3
3.1. Synchronization and Coordination Issues.....	3
3.1.1. Seat-Level Mutual Exclusion.....	3
3.1.2. Seat State Consistency and Timeout Race Conditions.....	3
3.1.3. Coordination Across Multiple Erlang Nodes.....	3
3.1.4. Java–Erlang Request/Response Coordination.....	3
3.2. Communication Issues.....	3
3.2.1. Java ↔ Erlang Protocol Definition.....	3
3.2.2. Erlang → Asynchronous Worker Notifications.....	4
3.2.3. Handling Communication Failures and Timeouts.....	4
4. System Overview.....	4
5. Architecture.....	4
6. Functional Requirements.....	5
6.1. User Operations.....	5
6.2. System Behavior Requirements.....	6
7. Non-Functional Requirements.....	6
8. Sequence Diagram.....	7
8.1. Seat Confirmation Flow.....	7
8.2. Hold Expiration Flow.....	7
8.3. Failover and Node Recovery.....	8
8.4. Concurrent Hold Requests (Race Condition Handling).....	9
8.5. Timeout Seat.....	10
9. Technologies.....	10
10. Implementation Details.....	10
10.1. Erlang Core.....	10
10.2. Spring Boot Gateway.....	11
10.3. Worker Service.....	11
10.4. Chat Consumer (Demo).....	11
11. Concurrency Control.....	11
12. Testing Strategy.....	11
13. Conclusion.....	12

Abstract

This project implements a distributed ticket reservation system that ensures consistent seat allocation under concurrent requests. The system uses Erlang and Mnesia for fault-tolerant, transactional seat management, with a Spring Boot (Java) gateway exposing REST APIs. A Kafka based event flow handles hold expirations and downstream notifications. The design demonstrates partitioned routing, transactional concurrency control, and event driven expiration handling.

1. Introduction

Modern ticketing platforms must process thousands of simultaneous reservation requests within strict timing constraints. Overselling or double-booking a seat produces significant financial and legal risks, and therefore requires strong synchronization mechanisms. In addition, real-world systems must react to timeouts, handle failures gracefully, and maintain consistent seat states under high concurrency.

This project explores a pragmatic, educational implementation that combines Erlang's lightweight concurrency and Mnesia's transactional store with a Spring Boot gateway and Kafka for event driven processing. The focus is on correctness under contention (atomic holds), automatic expiration, robust asynchronous communication between system components, and clear system boundaries that can be extended with additional services (e.g., notifications, UI).

1.1. Project Compliance with Course Requirements

This project fulfills the Distributed Systems And Middleware Technologies course requirements:

- **Multi-node deployment:** The system deploys across multiple nodes (3 Erlang nodes: res1, res2, res3; Spring Boot gateway, Kafka, Zookeeper, and worker services), demonstrating true distributed operation.
- **Erlang implementation:** Core reservation logic, Mnesia database management, and distributed coordination are implemented entirely in Erlang/OTP, showcasing supervision trees, gen_server behavior, and transactional operations.
- **Synchronization/coordination issues:** The project explicitly addresses seat-level mutual exclusion, timeout race conditions, multi-node coordination, and request/response correlation (detailed in Problem Statement section).
- **Communication issues:** Demonstrates inter-process communication via Erlang message passing, Java–Erlang interoperability through JInterface, and asynchronous event-driven architecture using Kafka.
- **Technology justification:** Erlang provides fault tolerance and lightweight concurrency; Mnesia offers distributed transactions; Spring Boot enables REST APIs; Kafka decouples notification processing; Docker ensures reproducible deployments.

2. Objectives

The architecture demonstrates the following design goals:

- Correct mutual exclusion for each seat under high concurrency
- Reliable timeout and state management with automatic hold expiration
- Robust asynchronous communication between system components
- Interoperability between Java and Erlang via JInterface
- Distributed coordination across multiple Erlang nodes
- Fault tolerance, leveraging Erlang's OTP supervision tree
- Expose a clean REST API for clients and provide reproducible tests

3. Problem Statement

3.1. Synchronization and Coordination Issues

3.1.1. Seat-Level Mutual Exclusion

Many users may concurrently attempt to hold the same seat. Without proper synchronization, two hold operations could both incorrectly succeed, resulting in inconsistent or duplicated reservations. The system must guarantee that only one user can successfully hold a seat at any given time.

3.1.2. Seat State Consistency and Timeout Race Conditions

Incorrect state transitions may occur if:

- A timeout fires at nearly the same moment a user sends a confirmation
- Multiple confirmation or hold requests overlap
- A stale timeout message arrives after the seat has transitioned to sold

Such situations may lead to invalid state changes and inconsistency. The system must maintain strict state machine semantics: **free** → **held** → **sold** or **free** → **held** → **free** (on timeout).

3.1.3. Coordination Across Multiple Erlang Nodes

If the system runs multiple Erlang nodes, the same seat must not be distributed across nodes or handled inconsistently. All operations for a specific event must reach the same reservation node to maintain atomicity and consistency. This requires a deterministic routing strategy based on seat identifiers.

3.1.4. Java–Erlang Request/Response Coordination

Because HTTP requests in Java are synchronous but Erlang responses are asynchronous, replies must be correctly matched to requests. Without correlation, responses may be mismatched, lost, or delayed indefinitely. The gateway must implement proper correlation mechanisms to pair requests with their corresponding Erlang responses.

3.2. Communication Issues

3.2.1. Java ↔ Erlang Protocol Definition

Java and Erlang operate on different runtimes and require explicit agreement on message structures. Without a defined protocol, misinterpretation or incompatibility may occur. The system uses JInterface to bridge this gap, establishing a clear message format for all operations.

3.2.2. Erlang → Asynchronous Worker Notifications

Timeout and confirmation events require processing outside the main reservation path. Such tasks should not delay seat management nor introduce new synchronization issues. The system decouples notification processing via Kafka, ensuring that downstream workers operate independently without blocking the critical reservation path.

3.2.3. Handling Communication Failures and Timeouts

Delays or failures between Java and Erlang or between Erlang and worker services may result in lost messages, duplicate processing, or client-visible failures. The architecture implements failover routing, retry mechanisms, and idempotent operations to handle transient network issues gracefully.

4. System Overview

The system consists of:

- Erlang core cluster (3 nodes) managing the distributed seat table in Mnesia.
- Java Spring Boot gateway that routes requests to Erlang nodes via JInterface and publishes expiration events to Kafka.
- Kafka + Zookeeper as the event backbone.
- Java worker service consuming Kafka events (simulated notifications).
- Seat UI (React + Vite + Nginx) for front-end exploration.

High-level flow:

1. Client calls REST endpoints on the gateway (hold, check, confirm).
2. Gateway routes to Erlang nodes based on seatId prefix (A/B → res1, C/D → res2, E/F → res3).
3. Erlang executes a transactional operation against Mnesia (seat table) to hold/confirm seats.
4. A cleanup task runs every 5 seconds to expire holds and emits hold_expired events.
5. Gateway publishes events to Kafka; a worker service consumes them for notifications.

5. Architecture

Services:

- **res1, res2, res3**: Erlang nodes sharing cookie ticketcookie; disc_copies replication in Mnesia.
- **gateway**: Spring Boot application exposing REST, integrating with Erlang via JInterface.
- **kafka, zookeeper**: Messaging infrastructure.
- **worker**: Java Kafka consumer (email simulation in logs).

Data model seat record attributes: [key, state, user_id, hold_id, expires_at, order_id]

Routing rules (primary):

- **State lifecycle:** Seat states must follow the defined lifecycle:
 - free → held → sold, or
 - free → held → free (timeout)
- **Gateway forwarding:** The Java gateway must forward user requests to the Erlang reservation core and return responses to clients based on Erlang's decisions.
- **Logging:** All communication between modules must be logged for debugging, testing, and transparency.
- **Multi-node support:** The architecture must support multiple Erlang reservation nodes executing concurrently.
- **Distributed routing:** Requests are routed to Erlang nodes by seat prefix (A/B → res1, C/D → res2, E/F → res3).
- **Failover handling:** If a primary node is unavailable, the gateway retries a deterministic fallback order.

7. Non-Functional Requirements

- **Consistency:** Use Mnesia transactions to ensure atomic read-check-write and prevent race conditions.
- **Availability:** Operate with three Erlang nodes; gateway implements retry/failover for transient node issues.
- **Scalability:** Horizontal scaling via seat-prefix partitioning across Erlang nodes; stateless HTTP gateway.
- **Performance:** Typical hold operation completes within low milliseconds on a warm system; expiration scan runs every 5 seconds.
- **Reliability:** Expired holds are reclaimed deterministically; events are delivered to Kafka for durable consumption.
- **Observability:** Structured logs for routing, concurrency, and expiration; containerized logs accessible via Docker.
- **Security:** Demo scope (no auth); to be enhanced with authentication/authorization and rate limiting in future work.
- **Portability:** Docker for reproducible local deployments across environments.

8. Sequence Diagram

8.1. Seat Confirmation Flow

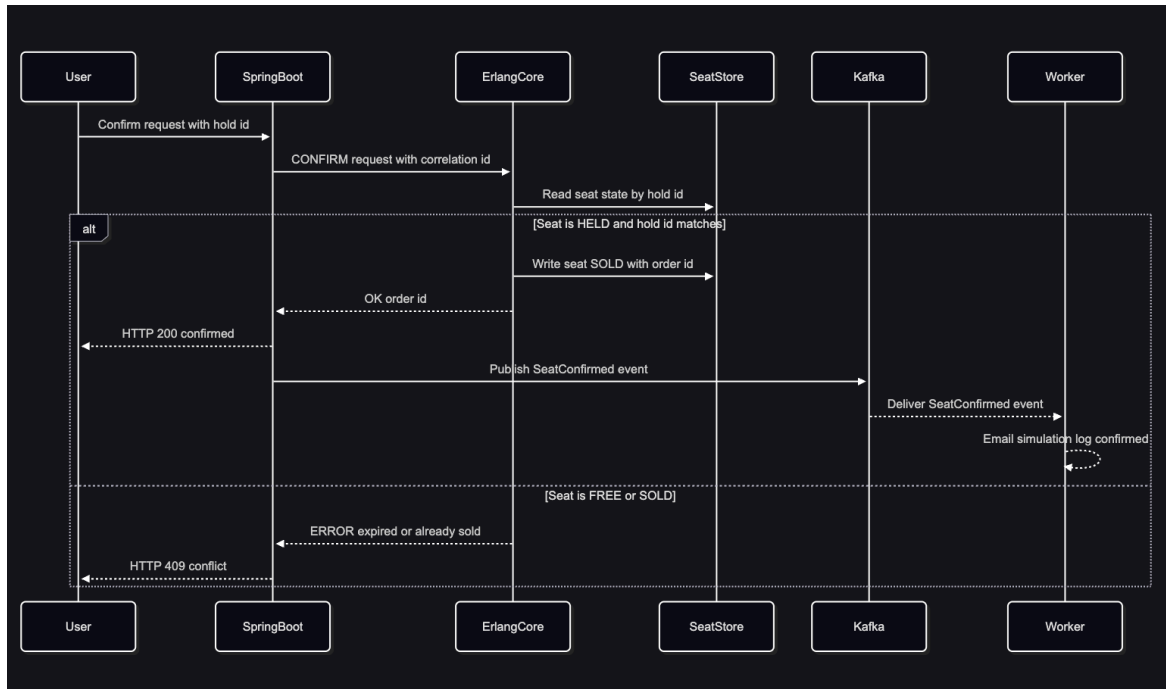


Figure 1: Complete confirmation workflow showing hold request validation, Mnesia transaction execution for seat purchase, Kafka event publishing, and worker notification processing.

8.2. Hold Expiration Flow

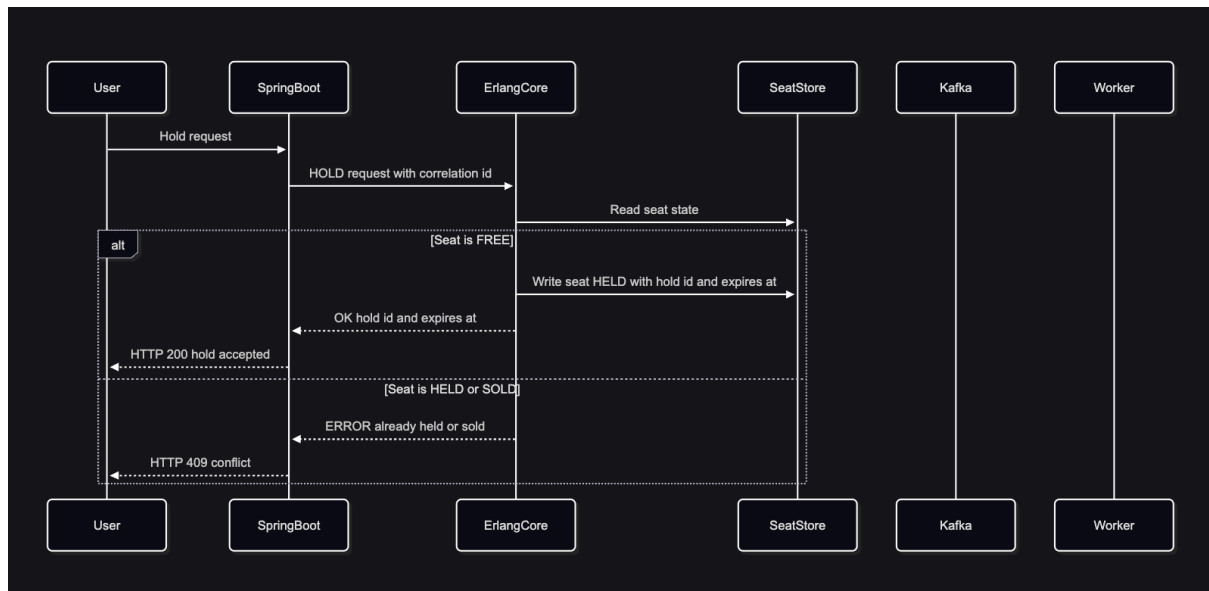


Figure 2: Automatic hold expiration workflow demonstrating the periodic cleanup task, seat state transition from HELD to FREE, and asynchronous notification delivery via Kafka.

8.3. Failover and Node Recovery

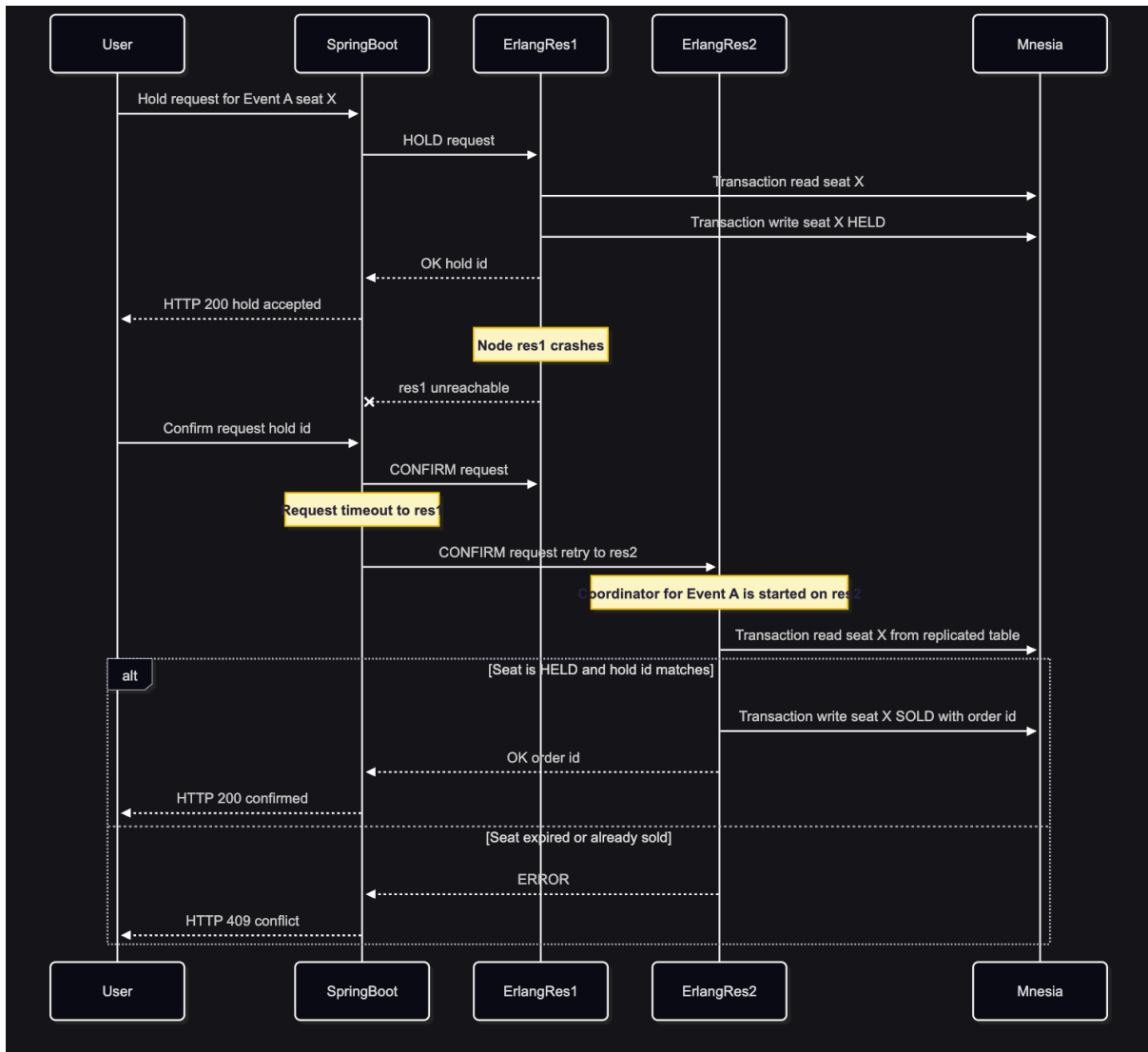


Figure 3: Distributed coordination with failover handling. When the primary Erlang node (res1) becomes unavailable, the gateway automatically retries the request to the secondary node (res2), demonstrating fault tolerance.

8.4. Concurrent Hold Requests (Race Condition Handling)

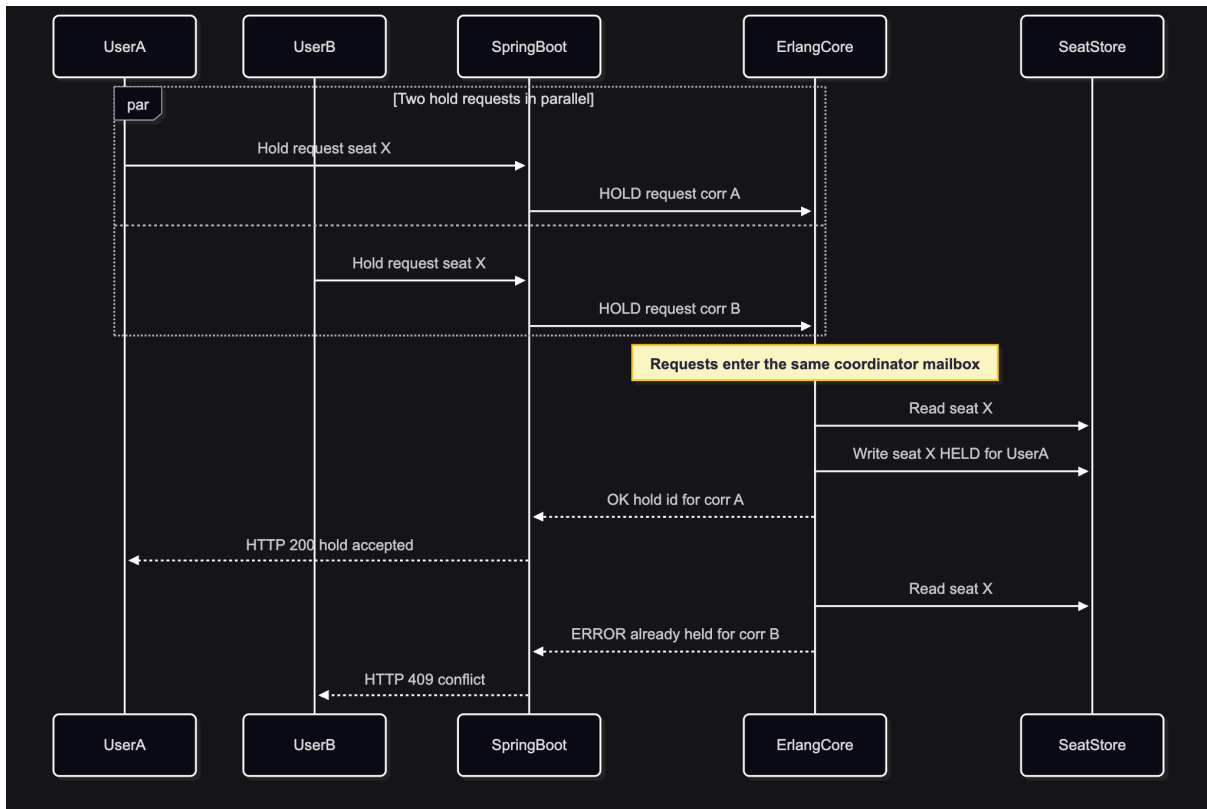


Figure 4: Concurrent seat hold requests from UserA and UserB. The coordinator's mailbox serializes requests, and Mnesia transactions ensure only one user successfully holds the seat while the other receives a conflict error.

8.5. Timeout Seat

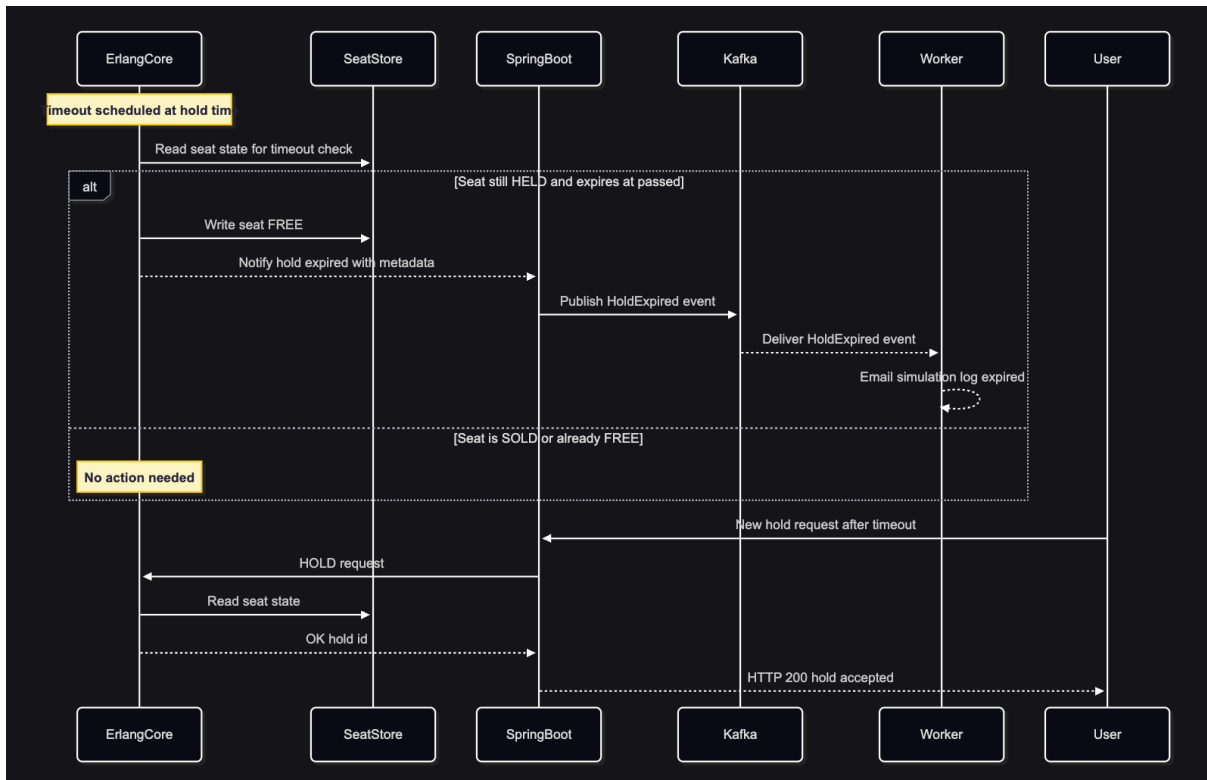


Figure 5: Seat timeout and expiration flow. The ErlangCore coordinator manages the transition of seat states in the SeatStore and triggers asynchronous expiration events through Kafka to ensure eventual consistency and system notification of released holds

9. Technologies

- Erlang/OTP, Mnesia (distributed transactions, fault tolerance)
- Java 21, Spring Boot, JInterface (Erlang interop)
- Kafka 7.6.0, Zookeeper
- Docker & Docker Compose v2
- React, Typescript

10. Implementation Details

10.1. Erlang Core

- **Bootstrap:** `init_mnesia.erl` creates Mnesia schema on `res1`, followers join after marker presence.
- **Supervision tree:** `reservation_core_sup.erl` starts `seat_srv.erl`.
- Server API (mailbox messages): `{hold_seat, ...}`, `{check_seat, ...}`, `{confirm_reservation, ...}` (representative).
- **Transactional logic:** `hold_tx` encapsulates read-check-write under `mnesia:transaction/1` to ensure atomicity.
- **Cleanup task:** periodic scan (5s) to transition expired holds to free and emit `hold_expired` messages.

10.2. Spring Boot Gateway

- **REST endpoints:**
 - POST /hold → create hold with expiration seconds
 - GET /check → query seat state (free, held, expired, sold)
 - POST /reservations/confirm → complete purchase
- **Routing:** calculates target Erlang node from seatId prefix; includes failover on ping/timeout.
- **Kafka integration:** publishes hold_expired events with metadata.

10.3. Worker Service

- Consumes Kafka topic (e.g., seat-hold-expired).
- Simulates notifications (email output in logs) with payload: eventId, seatId, holdId, expiresAtMillis, userId.
- Independent Java consumer (infra/chat-consumer-java) listening to confirmation/expiration topics.
- Example hook to Google Chat via webhook (for demo notifications).

11. Concurrency Control

- All hold operations run inside Mnesia transactions to serialize concurrent requests.
- First request to a free seat succeeds; later concurrent requests fail with {seat_already_held, ...}.
- Additional constraints:
 - Single active hold per user.
 - Sold seats cannot be held again.
 - Expired holds are reclaimed automatically.
- Logging prefix [CONCURRENCY] documents transaction steps for audit.

12. Testing Strategy

- Detailed, reproducible test cases documented in testing guide.
- **Basic flows:** HOLD → CHECK → CONFIRM → SOLD.
- **Expiration flow:** short holdSeconds then verify transition back to free and Kafka notification.
- **Concurrency tests:** automated script launching two simultaneous holds for the same seat; one succeeds, one fails.
- **Distributed routing tests:** verify A/B, C/D, E/F prefixes reach different nodes and log routing decisions.

13. Conclusion

This project demonstrates a compact yet robust design for seat reservations under contention, fulfilling all course requirements for a distributed software system.

Key achievements:

- **Multi-node distributed deployment:** Successfully deployed across 3 Erlang nodes plus additional service containers, demonstrating real distributed coordination.
- **Erlang core implementation:** Leveraged Erlang's concurrency model, OTP supervision, and Mnesia transactions to provide strong consistency for hold/confirm operations.
- **Synchronization solutions:** Addressed seat-level mutual exclusion via Mnesia transactions, preventing race conditions under concurrent load; resolved timeout race conditions through atomic state transitions.
- **Communication solutions:** Implemented Java–Erlang interoperability via JInterface with proper correlation mechanisms; decoupled notification processing through Kafka for reliable, asynchronous event delivery.
- **Working demonstration:** System includes automated tests, concurrency validation scripts, and reproducible Docker Compose deployment, ready for live demonstration.

The architecture scales by partitioning seat ranges across nodes and remains portable through Docker Compose. With authentication, persistent order storage, metrics, and a richer UI, this educational sandbox demonstrates the foundations for a production-grade distributed ticketing system.

Technology justifications:

- **Erlang/OTP:** Lightweight processes, built-in fault tolerance via supervision trees, and actor-model concurrency eliminate complex locking mechanisms.
- **Mnesia:** Distributed transactional database with seamless Erlang integration, providing ACID guarantees without external dependencies.
- **Spring Boot + JInterface:** Bridges HTTP clients with Erlang backend, enabling synchronous REST APIs over asynchronous Erlang messaging.
- **Kafka:** Decouples time-sensitive reservation operations from notification delivery, ensuring seat management remains responsive.
- **Docker Compose:** Ensures reproducible multi-node deployments across development environments.